

งานของผู้เรียน – ระบบชมรมด้วย Neo4j

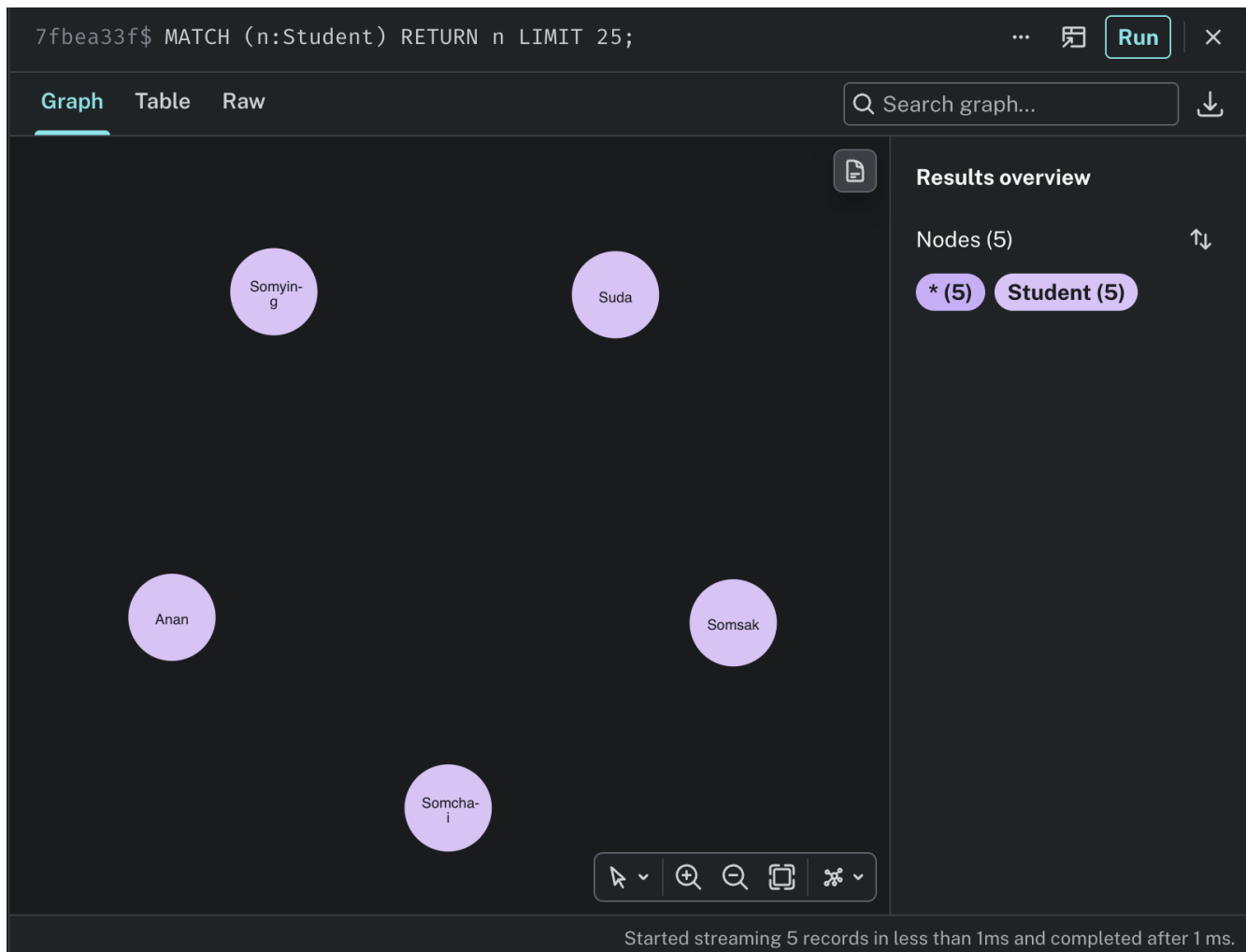
คำสั่ง Cypher พร้อมพื้นที่สำหรับแปปหน้าจอตผลลัพธ์

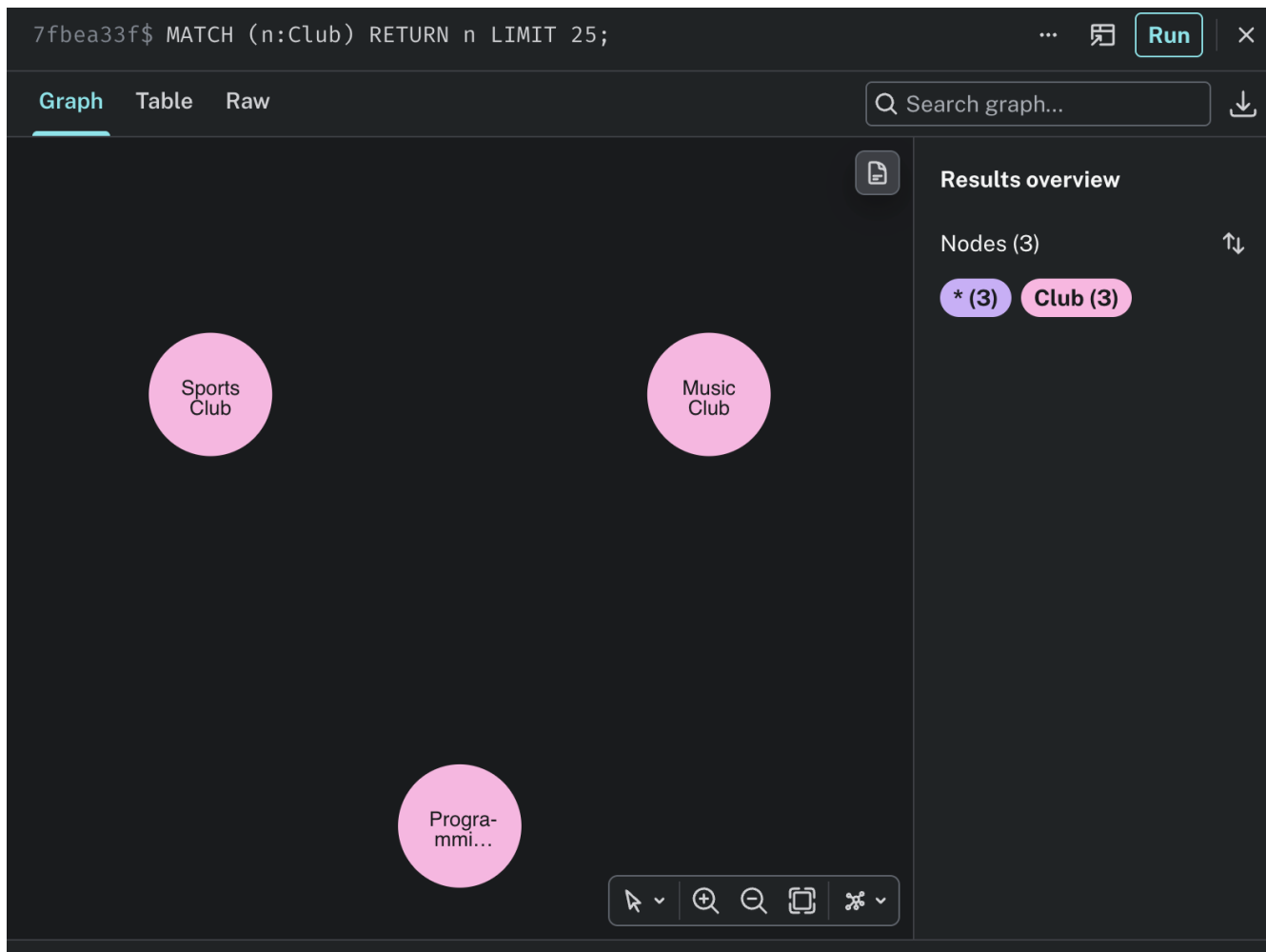
1. สร้าง Student 5 คน และ Club 3 ชมรม

สร้างข้อมูลตัวอย่างสำหรับระบบชมรม โดยมี Student อย่างน้อย 5 คน และ Club 3 ชมรม

```
CREATE
(s001:Student {id:'S001', name:'Somchai'}),
(s002:Student {id:'S002', name:'Somsak'}),
(s003:Student {id:'S003', name:'Suda'}),
(s004:Student {id:'S004', name:'Somying'}),
(s005:Student {id:'S005', name:'Anan'}),

(c001:Club {id:'C001', name:'Programming Club'}),
(c002:Club {id:'C002', name:'Music Club'}),
(c003:Club {id:'C003', name:'Sports Club'});
```

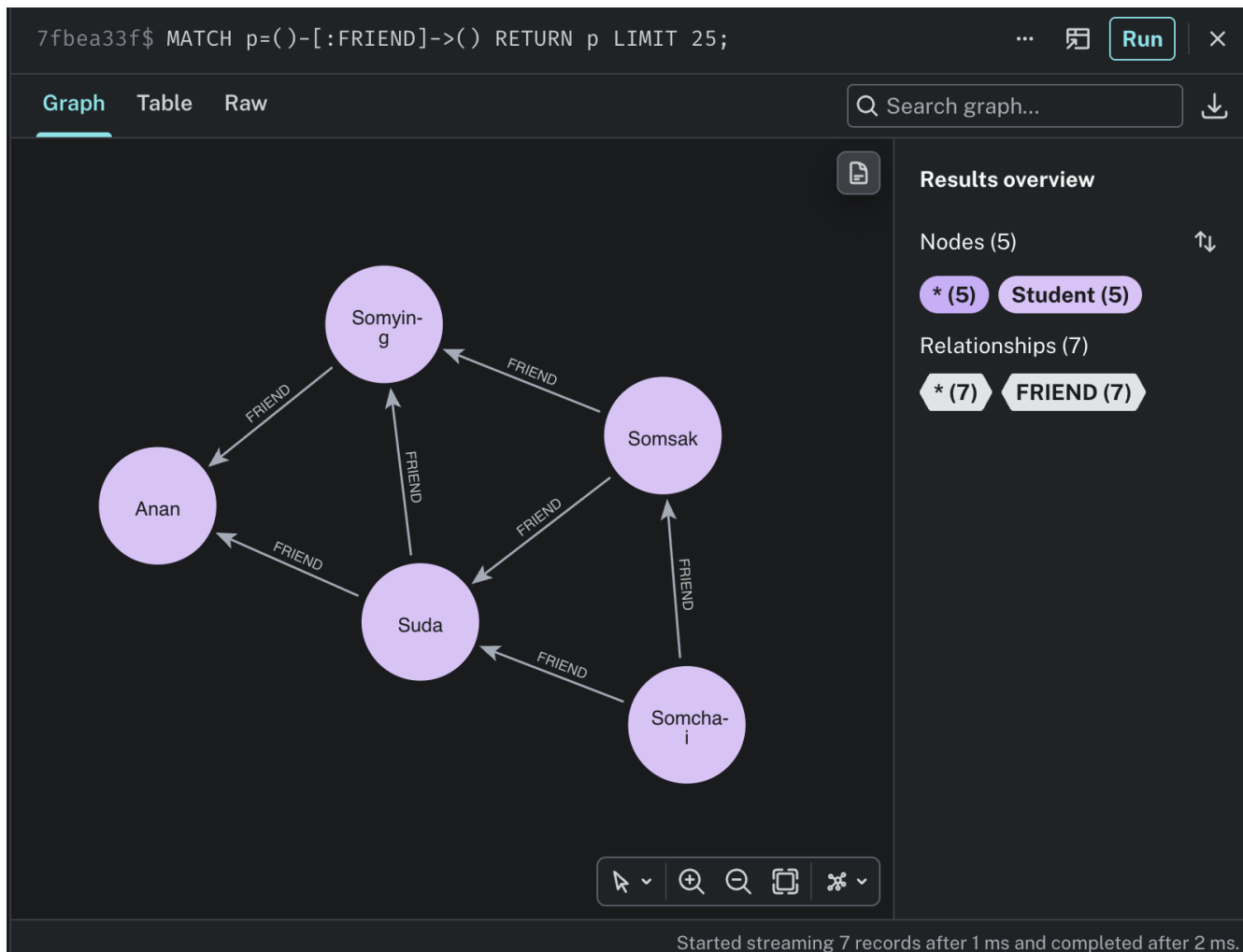




2. สร้าง Relationship FRIEND

กำหนดความสัมพันธ์ระหว่างนักศึกษา เพื่อใช้ในการ Traversal และ Recommendation

```
MATCH
(s001:Student {id:'S001'}),
(s002:Student {id:'S002'}),
(s003:Student {id:'S003'}),
(s004:Student {id:'S004'}),
(s005:Student {id:'S005'})
CREATE
(s001)-[:FRIEND]->(s002),
(s001)-[:FRIEND]->(s003),
(s002)-[:FRIEND]->(s003),
(s002)-[:FRIEND]->(s004),
(s003)-[:FRIEND]->(s004),
(s003)-[:FRIEND]->(s005),
(s004)-[:FRIEND]->(s005);
```



แดปหน้าจอผลลัพธ์จาก Neo4j ในเครื่องนี้

3. สร้าง Relationship MEMBER_OF

กำหนดว่านักศึกษาแต่ละคนเป็นสมาชิกของชมรมใด

```
MATCH
(s001:Student {id:'S001'}),
(s002:Student {id:'S002'}),
(s003:Student {id:'S003'}),
(s004:Student {id:'S004'}),
(s005:Student {id:'S005'}),
(c001:Club {id:'C001'}),
(c002:Club {id:'C002'}),
(c003:Club {id:'C003'})
CREATE
(s001)-[:MEMBER_OF]->(c001),
(s001)-[:MEMBER_OF]->(c002),
```

```

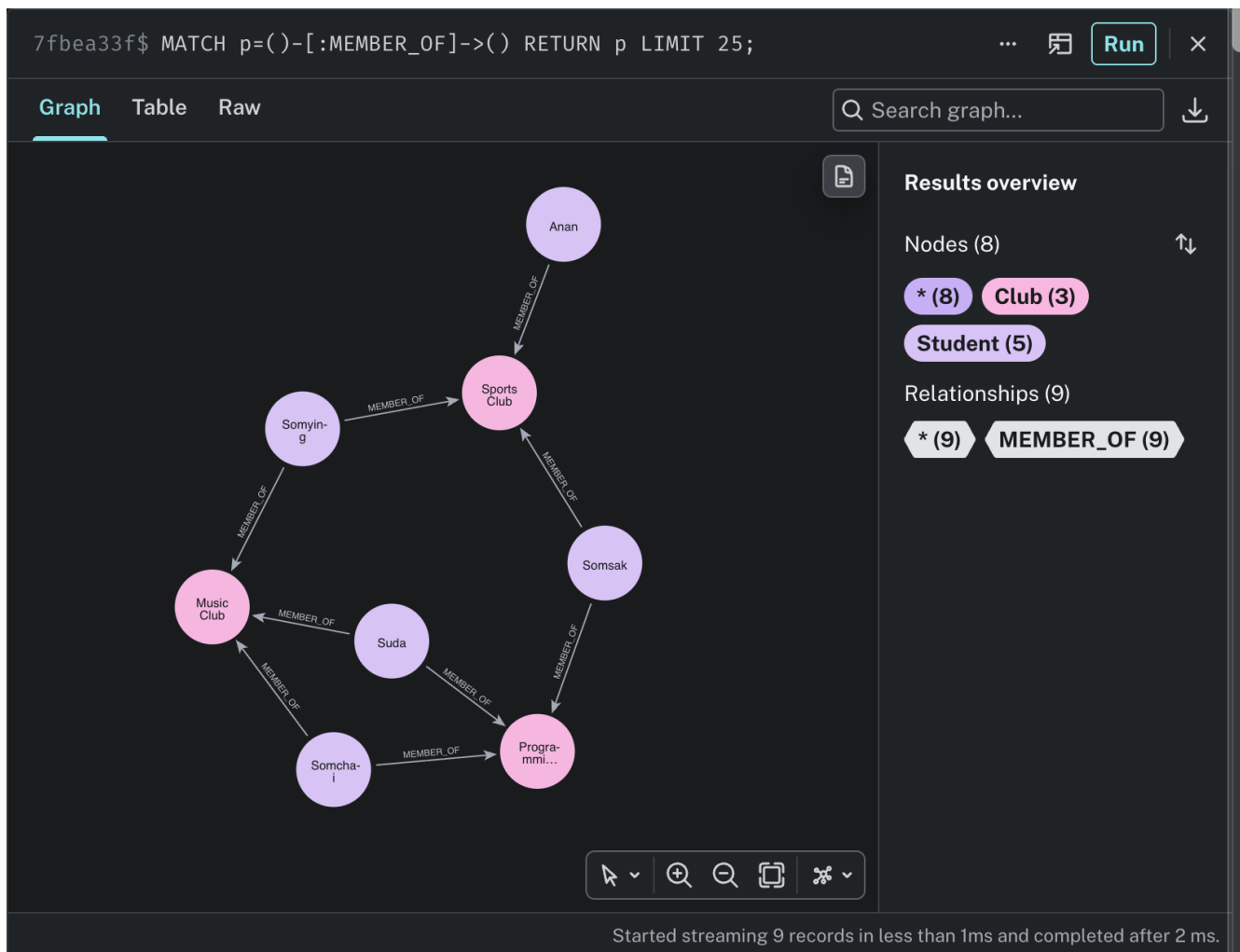
(s002)-[:MEMBER_OF]->(c001),
(s002)-[:MEMBER_OF]->(c003),

(s003)-[:MEMBER_OF]->(c001),
(s003)-[:MEMBER_OF]->(c002),

(s004)-[:MEMBER_OF]->(c002),
(s004)-[:MEMBER_OF]->(c003),

(s005)-[:MEMBER_OF]->(c003);

```



4. ใช้ MATCH + WHERE + RETURN ดูชมรมของ S001

ค้นหาชมรมที่ Student S001 เป็นสมาชิก โดยใช้ MATCH, WHERE และ RETURN ตามโจทย์

```

MATCH (s:Student)-[:MEMBER_OF]->(c:Club)
WHERE s.id = 'S001'
RETURN c.id AS ClubID, c.name AS ClubName;

```

Table	Raw	Q Search table...	Graph chip	Download	Copy
ClubID	ClubName				
1 "C001"	"Programming Club"				
2 "C002"	"Music Club"				

5. ใช้ Traversal ไปยัง Club

เริ่มจาก S001 → FRIEND → เพื่อน → MEMBER_OF → Club เพื่อดูว่าเพื่อนของ S001 อยู่ชมรมใดบ้าง

```
MATCH (s:Student {id:'S001'})
  -[:FRIEND]->(friend:Student)
  -[:MEMBER_OF]->(club:Club)
RETURN friend.name AS Friend, club.name AS Club;
```

Table	Raw	Q Search table...	Graph chip	Download	Copy
Friend	Club				
1 "Somsak"	"Programming Club"				
2 "Somsak"	"Sports Club"				
3 "Suda"	"Programming Club"				
4 "Suda"	"Music Club"				

6. ใช้ count(DISTINCT friend)

นับจำนวนเพื่อนของ S001 ที่อยู่ในแต่ละ Club และเรียงจากจำนวนมากไปน้อย

```
MATCH (s:Student {id:'S001'})
  -[:FRIEND]->(friend:Student)
  -[:MEMBER_OF]->(club:Club)
RETURN club.name AS Club,
  count(DISTINCT friend) AS FriendCount
ORDER BY FriendCount DESC;
```

Table	Raw	Q Search table...	Graph chip	Download	Copy
Club	FriendCount				
1 "Programming Club"	2				
2 "Sports Club"	1				
3 "Music Club"	1				Copy

7. ตัด Club ที่ S001 เป็นสมาชิกอยู่แล้ว

ใช้ WHERE NOT เพื่อตัดขมรมที่ S001 เป็น MEMBER_OF อยู่แล้วออกจากผลลัพธ์

```
MATCH (s:Student {id:'S001'})
      -[:FRIEND]->(friend:Student)
      -[:MEMBER_OF]->(club:Club)
WHERE NOT (s)-[:MEMBER_OF]->(club)
RETURN club.name AS Club,
       count(DISTINCT friend) AS FriendCount
ORDER BY FriendCount DESC;
```

Table	Raw	Q Search table...	Graph chip	Download	Copy
Club	FriendCount				
1 "Sports Club"	1				

8. สร้าง Recommendation อันดับ 1

แนะนำ Club ที่มีเพื่อนของ S001 เป็นสมาชิกมากที่สุด โดยตัดขมรมที่ S001 เป็นสมาชิกอยู่แล้ว และเลือกอันดับ 1 ด้วย LIMIT 1

```
MATCH (s:Student {id:'S001'})
      -[:FRIEND]->(friend:Student)
      -[:MEMBER_OF]->(club:Club)
WHERE NOT (s)-[:MEMBER_OF]->(club)
RETURN club.name AS RecommendedClub,
       count(DISTINCT friend) AS FriendCount
ORDER BY FriendCount DESC
```

```
LIMIT 1;
```

Table Raw		Q Search table...	Graph chip ▾	↓ ▾	📄 ▾
RecommendedClub		FriendCount			
1	"Sports Club"	1			

คำอธิบายเส้นทางความสัมพันธ์

ระบบเริ่มจาก Student S001 แล้ว Traversal ผ่านความสัมพันธ์ FRIEND ไปยังเพื่อน จากนั้นเดินต่อผ่าน MEMBER_OF ไปยัง Club และใช้ count(DISTINCT friend) เพื่อนับจำนวนเพื่อนในแต่ละชมรม จากนั้นใช้ WHERE NOT ตัดชมรมที่ S001 เป็นสมาชิกอยู่แล้ว และเรียงจำนวนเพื่อนจากมากไปน้อยเพื่อเลือก Recommendation อันดับ

1